



ASSIGNMENT-2

Part 1 — System Design

The Problem This Portal Solves

Before this portal, E-Cell members reported issues over WhatsApp and email. Messages got lost, nobody tracked whether anything was fixed, and there was no clear record of who was responsible. This portal puts everything in one organised place.

How a Ticket Moves Through the System

The diagram below shows the full journey from the moment a user submits a ticket to the moment it is resolved.



The 5 Ticket Stages

Every ticket has a status that tells everyone exactly where it stands. The table below explains each stage, what it means, and who is allowed to move the ticket to that stage.

Status	What it means	Who sets it	Next possible stage
OPEN	Ticket just submitted. Nobody has picked it up yet.	Automatic on creation	ASSIGNED
ASSIGNED	A coordinator has been given responsibility for this ticket.	Manager or OC	IN PROGRESS
IN PROGRESS	The coordinator is actively working on fixing the issue.	Coordinator or Manager	RESOLVED

RESOLVED	Issue is fixed and the ticket is closed.	Coordinator or Manager	None — final stage
OVERDUE	Ticket sat OPEN for more than 24 hours without being assigned.	System sets automatically	ASSIGNED (after OC steps in)

What a Ticket Contains

Each ticket stores structured information so there is no need to ask follow-up questions or search through email chains.

Field	What it stores	Why it is needed
Title	Short one-line summary of the issue	Makes the ticket scannable in a list
Description	Full details, steps to reproduce the problem	Gives the coordinator all the context upfront
Priority	LOW / MEDIUM / HIGH / CRITICAL	Helps the team focus on urgent issues first
Status	Current lifecycle stage	Everyone knows where the ticket stands at a glance
Department	Which team is responsible	Routes the ticket to the right people
Assignee	The coordinator handling it	One person is clearly accountable
Tags	Keywords like payment, mobile, urgent	Makes filtering and searching faster
Attachments	Screenshots, files, documents	Evidence is attached, not sent separately over email
ID (UUID)	A random unique identifier	Prevents anyone from guessing ticket IDs

How File Attachments Are Handled

Users can upload screenshots and files directly on a ticket. Here is exactly what happens at each step.

Step	Action	Detail
1	User picks a file	Sent to the server as multipart form data
2	Server checks file type	Images go to the image field. Other files go to the file field
3	File saved to disk	Stored under /media/ organised by year and month
4	Metadata saved to database	File name, size, type, uploader, and timestamp are recorded

5	Cleanup on delete	If the ticket is deleted, the file is automatically removed from disk too
---	-------------------	---

Part 2 — Access Control

The 4-Level Hierarchy

The system has four user roles arranged in a hierarchy. Each level can only do what their role permits and can only see data relevant to them. This is enforced on the server, not just in the interface.

OC (Organizing Committee) Maximum 2 people Sees all tickets Creates departments Promotes any user Receives escalation alerts
MANAGER One per department Sees tickets in their dept only Assigns coordinators Promotes users to coordinator
COORDINATOR Sees only assigned tickets Updates status Replies to users Can write internal notes
USER Creates tickets Views only their own tickets Chats on their own tickets Uploads attachments

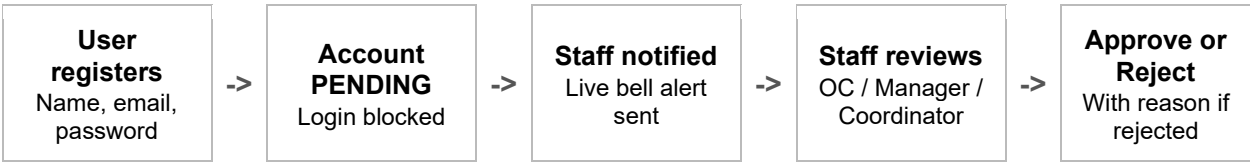
Who Can See and Do What

The table below is a quick reference. Y means allowed. N means not allowed.

Action	OC	Manager	Coordinator	User
See all tickets	Y — all	Y — own dept	Y — assigned only	Y — own tickets
Create departments	Y	N	N	N
Assign tickets	Y	Y	N	N
Change ticket status	Y	Y	Y	N
Promote users	Y	Y — own dept	N	N
Read internal notes	Y	Y	Y	N — hidden
Approve registrations	Y	Y	Y	N
View audit log	Y	N	N	N

New User Registration and Approval

Anyone can register but they cannot log in until a staff member approves them. This prevents random people from gaining access.



On approval the account is activated immediately and the user can log in. If rejected, they see the reason on their screen.

Three Layers That Prevent Unauthorized Access

Even if one layer is bypassed, the other two still protect the system.

No.	Layer	What it stops
1	JWT Token Check	No valid token means no access to any endpoint at all
2	Role Check on Every Route	Wrong role gets a 403 Forbidden response immediately
3	Database Query Filtering	Queries only return data the user owns or is allowed to manage, regardless of what they request

Part 4 — Additional Feature: Real-Time Chat with WebSockets

What This Feature Does

When a user opens a ticket and a coordinator types a reply, that message appears on the user's screen the instant it is sent. There is no need to refresh the page. Status changes also update live for everyone looking at the same ticket. The notification bell in the sidebar also lights up in real time when something relevant happens to any of your tickets.

Without real-time (old way)	With WebSockets (this portal)
User must refresh the page to check for replies Nobody knows when the status changed Coordinators send duplicate replies not knowing the user already responded	Messages appear instantly without any page refresh Status badge updates live on everyone's screen Typing indicator shows when the other person is composing a reply

Simple Explanation of How WebSockets Work

A normal web request is like posting a letter. You send it, get one reply, and that is it. A WebSocket is like a phone call that stays open. Both sides can send messages whenever they want, and the line stays open until one side hangs up.

Normal HTTP (one request, one reply)	WebSocket (open line, both sides talk freely)
Browser sends request to server Server sends one response back Connection is closed To get new data you must refresh the page	Browser and server open a connection once Server can push data to browser anytime Browser can send data anytime too Both sides stay in sync automatically

The Two WebSocket Channels

The portal runs two types of WebSocket connections at the same time.

Channel	What it carries	Who connects to it
Ticket Chat Channel <code>/ws/tickets/{id}/</code>	Chat messages, file attachments, status updates, typing indicators — all live inside one ticket	Anyone currently viewing that specific ticket page
Notifications Channel <code>/ws/notifications/</code>	Personal alerts — ticket assigned, account approved, escalation warning, new registration request	Every logged-in user has their own private channel that nobody else can read

How the Pieces Connect

Redis is the backbone that makes real-time work in production. The server can run as multiple processes. Redis connects all of them so a message sent on one process is delivered by another.



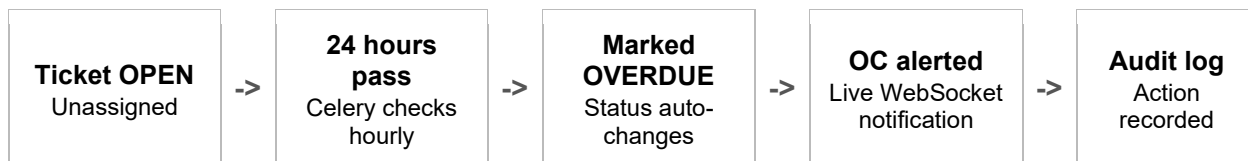
Security — Three Checks Before Any Message Is Delivered

The WebSocket connection is just as protected as any other part of the system.

	Check	What happens if it fails
1	JWT token in the URL is valid and not expired	Connection is rejected before it opens. Code 4001.
2	User has access to this specific ticket	Connection rejected. Code 4003. User sees an error message.
3	Internal note flag on each message	If the connected user is a regular User, internal notes are dropped on the server before delivery. The client never receives them.

Auto-Escalation — Nothing Slips Through

A background job runs every hour. It finds any ticket that has been sitting OPEN for more than 24 hours without being assigned. It marks it as OVERDUE and sends an instant alert to all OC members through their personal WebSocket notification channel.



This means even if a manager forgets to check the queue for a day, the OC will be alerted and can step in. The system never lets things go unnoticed silently.